SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0619 Design and Analysis of Algorithms

ASSIGNMENT

Assignment 2 – CO2

CO2 Coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort,

Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest

Pair, Convex Hull and Exhaustive Search.

Course Outcome: CO2 – Apply Brute Force technique to solve sorting, searching,

Closest-Pair and Convex-Hull problems

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 02.11.2026

Name:P.Ranjithkumar

Reg.no:192411119

INTRODUCTION AND EXPERIMENTAL METHOD

This report answers the seven questions in the supplied CSA06 Assignment 2 – CO2. The focus is not only on obtaining the correct output, but also on predicting behaviour, designing test data, counting important operations, explaining growth and making an engineering decision.

Common measurement rules

- Comparison count means comparisons between keys, characters, coordinates or candidate values, as applicable. Loop-control checks are not counted unless stated.
- Swap count records exchanges of two array elements. Movement/shift count records an element copied one position to the right during insertion sort.
- For a geometric distance calculation, one examined unordered pair produces one squared-distance calculation. Square roots are not required to compare distances.
- Execution time is machine-dependent. Operation counts are the primary evidence because they are deterministic for a fixed input and implementation.
- All sorting examples arrange scores in ascending order. For descending priority, reverse the comparison operators.

Complexity notation

Notation	Meaning
O(1)	Constant work, independent of input size
O(log n)	Work grows logarithmically
O(n)	Work grows proportionally to input size
O(n log n)	Typical efficient comparison-sorting growth
$O(n^2)$	Quadratic growth; work is proportional to n squared
$O(2^n)$	Exponential growth; each new item approximately doubles the search space

The examples below are deliberately small enough to verify by hand. Larger-size tables use exact mathematical operation counts where the algorithm's loop structure determines the count.

Q1 – SORTING UNDER DIFFERENT DATA CONDITIONS (6 MARKS)

1.1 Problem understanding and objective

The fulfilment centre must arrange order records using a dispatch priority score. The three candidate algorithms are Selection Sort, Bubble Sort and Insertion Sort. The important question is whether their work changes when the input is sorted, nearly sorted or reversed.

1.2 Prediction before implementation

Input arrangement	Selection Sort prediction	Bubble Sort prediction	Insertion Sort prediction
Already sorted	Still performs n(n−1)/2 comparisons; very few swaps	With early-stop flag, one pass and n−1 comparisons	Best case: n−1 comparisons and zero shifts
Nearly sorted	Comparison count remains quadratic; swaps may be low	Usually close to linear if only a few inversions exist	Few shifts; close to linear
Reverse order	Quadratic comparisons and many swaps	Quadratic comparisons and maximum adjacent swaps	Quadratic comparisons and maximum shifts

Reason: Selection Sort always scans the unsorted suffix to find its minimum. Bubble Sort can stop early when a pass makes no swaps. Insertion Sort inserts each next key into the already processed prefix; it is efficient when each key moves only a short distance.

1.3 Designed test datasets

Test	Input scores	Purpose
A – already sorted	10, 20, 30, 40, 50, 60, 70, 80	Tests best-case behaviour for adaptive algorithms
B – nearly sorted	10, 20, 30, 50, 40, 60, 70, 80	Only one adjacent inversion; exposes adaptive behaviour
C – reverse order	80, 70, 60, 50, 40, 30, 20, 10	Creates a high number of inversions and movements

1.4 Algorithm design

Selection Sort: for each position i, scan positions i+1 through n−1, remember the index of the smallest key, and swap it with position i. Bubble Sort: compare adjacent elements and exchange them when out of order; stop if a complete pass makes no exchange. Insertion Sort: take the next key, shift larger keys one position right, and insert the key into the gap.

1.5 Python implementation with operation counters

```
def selection_sort(a):
    a = a[:]; comparisons = swaps = 0
    for i in range(len(a) - 1):
        minimum = i
        for j in range(i + 1, len(a)):
            comparisons += 1
            if a[j] < a[minimum]:
                minimum = j
        if minimum != i:
            a[i], a[minimum] = a[minimum], a[i]
            swaps += 1
    return a, comparisons, swaps

def bubble_sort(a):
    a = a[:]; comparisons = swaps = 0
    for end in range(len(a) - 1, 0, -1):
        changed = False
        for j in range(end):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1; changed = True
        if not changed:
            break
    return a, comparisons, swaps

def insertion_sort(a):
    a = a[:]; comparisons = shifts = 0
    for i in range(1, len(a)):
        key = a[i]; j = i - 1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j + 1] = a[j]
                shifts += 1; j -= 1
            else:
                break
        a[j + 1] = key
```

```
    return a, comparisons, shifts

data = [10, 20, 30, 50, 40, 60, 70, 80]
print("Selection:", selection_sort(data))
print("Bubble:", bubble_sort(data))
print("Insertion:", insertion_sort(data))
```

1.6 Experimental operation-count table

Dataset	Selection comparisons	Selection swaps	Bubble comparisons	Bubble swaps	Insertion comparisons	Insertion shifts
Already sorted	28	0	7	0	7	0
Nearly sorted	28	1	13	1	8	1
Reverse order	28	4	28	28	28	28

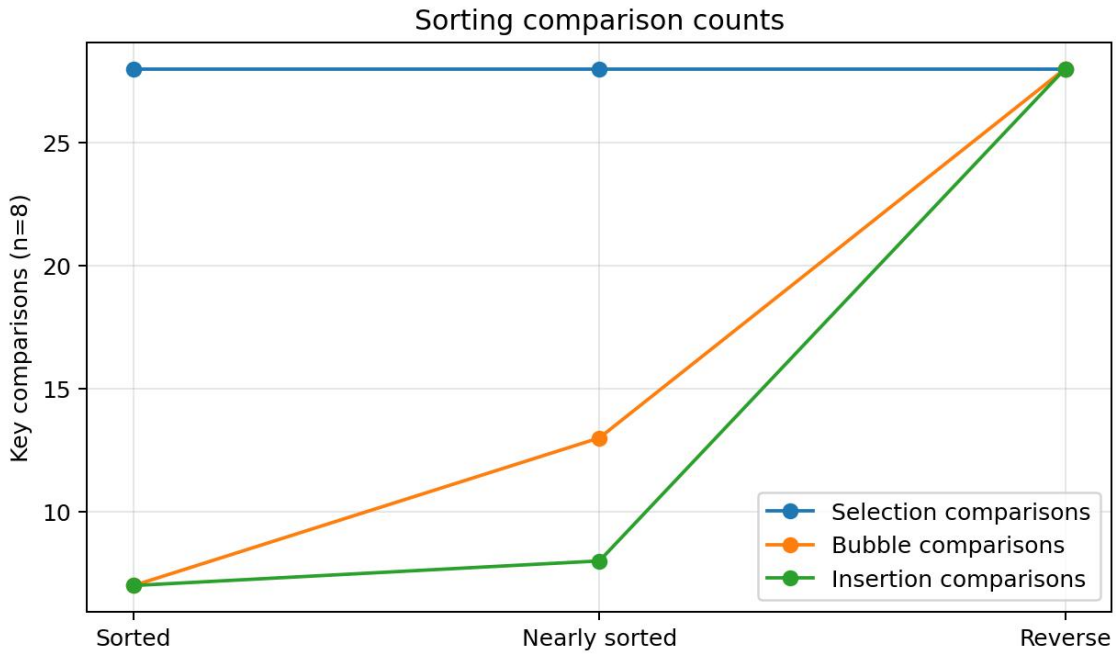


Figure 1. Comparison counts for the three algorithms on the designed n = 8 inputs.

1.7 Interpretation and recommendation

On sorted input, Selection Sort still makes 28 key comparisons for n = 8, while Bubble Sort with early stopping and Insertion Sort each make only 7 comparisons. On the reverse input, Insertion Sort shifts 28 elements and Bubble Sort performs 28 swaps. Selection Sort uses at most 7 swaps but still makes 28 comparisons.

Recommendation: Insertion Sort is the best fit among the three for this stated workload. Approximately 95% of existing orders are already in priority order and only small batches are added. Insertion Sort can place each new score into the correct position with a small number of shifts when the existing sequence is ordered. If the complete list is repeatedly re-sorted and is not nearly ordered, a more scalable sorting method would be considered; within the three choices, however, Insertion Sort best exploits the given input property.

Conclusion: no algorithm is always best. Selection Sort has predictable comparison count and low swap count; Bubble Sort benefits from early stopping; Insertion Sort is particularly effective for small or nearly sorted inputs.

Q2 – WHEN IS SEQUENTIAL SEARCH APPROPRIATE? (4 MARKS)

2.1 Problem understanding

Each active tool record stores Tool ID, Technician ID, Issue Time and Work Bay. The query asks for the technician currently holding a particular tool. Sequential Search checks records from the first entry until the requested Tool ID is found or the list ends.

2.2 Search procedure

```
def sequential_search(records, target):
    comparisons = 0
    for index, record in enumerate(records):
        comparisons += 1
        if record["tool_id"] == target:
            return index, record, comparisons
    return -1, None, comparisons

tools = [
    {"tool_id":"T-101", "technician":"TECH-1", "bay":"A1"},
    {"tool_id":"T-184", "technician":"TECH-7", "bay":"B3"},
    {"tool_id":"T-220", "technician":"TECH-4", "bay":"C2"}
]
print(sequential_search(tools, "T-184"))
print(sequential_search(tools, "T-999"))
```

2.3 Designed experiments and comparison counts

List size n	Successful at first position	Successful near middle	Successful at last position	Unsuccessful
10	1	5	10	10
100	1	50	100	100
1000	1	500	1000	1000

For a successful search at zero-based position k, comparisons = k + 1. For an unsuccessful search, every record must be checked, so comparisons = n. If a successful target is equally likely to be at any position, average comparisons = (n + 1) / 2.

2.4 Complexity analysis

Case	Comparisons	Time complexity
Best case: first record matches	1	O(1)
Average successful search, uniform positions	(n+1)/2	O(n)
Worst case: last record or not present	n	O(n)

2.5 Engineering decision

Sequential Search is reasonable when the active list is small, searches are infrequent, records are frequently inserted/deleted, and no index or sorting structure is maintained. It is also simple and has almost no preprocessing cost.

I would stop recommending it when the list becomes large and many lookups are requested. In the changed condition of 20 facilities, both the number of records and query frequency increase. Repeated scans can then perform hundreds or thousands of comparisons per request. A hash table indexed by Tool ID is a suitable redesign for exact ID lookup: it uses extra memory and maintenance work, but expected lookup time is approximately O(1). If ordered range queries are also needed, a balanced search tree or database index may be more appropriate.

2.6 Result

For a temporary list of a few dozen tools and occasional requests, Sequential Search is acceptable. For a combined multi-facility inventory with frequent lookups, use an indexed structure. The decision is based on total workload, not list size alone.

Q3 – ENGINEER THE BEHAVIOUR OF BRUTE-FORCE STRING MATCHING (6 MARKS)

3.1 Problem understanding

The security centre scans a text log for a known signature pattern. Brute-force matching aligns the pattern at every possible starting position and compares characters from left to right. If a mismatch occurs, it shifts the pattern by one character.

3.2 Pseudocode

```
BRUTE-FORCE-MATCH(text, pattern):
    comparisons = 0
    for shift = 0 to length(text) - length(pattern):
        j = 0
        while j < length(pattern):
            comparisons = comparisons + 1
            if text[shift + j] != pattern[j]:
                break
            j = j + 1
        if j == length(pattern):
            report shift
    return all matches and comparisons
```

3.3 Three deliberately designed tests

Test condition	Text	Pattern	Prediction
A – early mismatch	ZX followed by 30 A characters	YYZZ	First character mismatches at most shifts; very little work
B – repeated-prefix case	30 A characters followed by B	10 A characters followed by C	Each alignment matches many A characters and fails near the end
C – actual signature	LOG:WARN:AUTH:FAIL:END	AUTH:FAIL	A successful match occurs after several shifts

3.4 Implementation and measured character comparisons

```
def brute_force_match(text, pattern):
    matches = []
    comparisons = 0
    if pattern == "":
        return list(range(len(text) + 1)), 0
    for shift in range(len(text) - len(pattern) + 1):
        j = 0
        while j < len(pattern):
            comparisons += 1
            if text[shift + j] != pattern[j]:
                break
            j += 1
        if j == len(pattern):
            matches.append(shift)
    return matches, comparisons

tests = [
    ("ZX" + "A"*30, "YYZZ"),
    ("A"*30 + "B", "A"*10 + "C"),
    ("LOG:WARN:AUTH:FAIL:END", "AUTH:FAIL")
]
for text, pattern in tests:
    print(brute_force_match(text, pattern))
```

Test	Text length	Pattern length	Character comparisons	Match positions
Early mismatch	32	4	29	[]
Repeated-prefix worst case	31	11	231	[]
Pattern found	22	9	23	[9]

3.5 Why the effort differs

Input length alone does not determine the exact number of comparisons. The pattern and text character arrangement also matter. In Test A, mismatches occur at the first character, so most alignments stop immediately. In Test B, the text and pattern share a long prefix of A characters. The algorithm repeats those successful comparisons at every one-character shift before discovering the final mismatch. In Test C, the algorithm must scan preceding alignments before it reaches the signature.

For text length n and pattern length m, there are $n - m + 1$ possible alignments. In the worst case, each alignment compares m characters, giving $O((n - m + 1)m)$, commonly written $O(nm)$. The best case can be close to $O(n)$ when each alignment fails immediately.

3.6 Conclusion

Brute-force string matching is simple and suitable for small texts or short patterns. For large log files with repetitive content and long signatures, repeated prefix comparisons can be expensive. The next question redesign uses a prefix table to reuse information from earlier comparisons.

Q4 – CLOSEST PAIR AND THE COST OF EXAMINING EVERY POSSIBILITY (6 MARKS)

4.1 Mathematical derivation of candidate pairs

A closest pair must be one of the unordered pairs of distinct drones. For n drones, choose any 2 drones from n:

$C(n, 2) = n! / (2!(n-2)!) = n(n-1) / 2$

To avoid duplicate comparisons, use indices i and j with j > i. Thus (D1,D2) is examined once and the reversed pair (D2,D1) is never examined.

4.2 Distance calculation

$d(D_i,D_j) = \sqrt{[(x_i - x_j)^2 + (y_i - y_j)^2]}$

The square root is unnecessary while selecting the minimum. Compare squared distances: $(x_i - x_j)^2 + (y_i - y_j)^2$. This gives the same closest pair and avoids repeated square-root calculations.

4.3 Algorithm and Python implementation

```
def closest_pair_bruteforce(points):
    n = len(points)
    if n < 2:
        raise ValueError("At least two points are required")
    best_pair = None
    best_d2 = float("inf")
    pairs_examined = 0
    distance_calculations = 0

    for i in range(n):
        for j in range(i + 1, n):
            pairs_examined += 1
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]
            d2 = dx*dx + dy*dy
            distance_calculations += 1
            if d2 < best_d2:
                best_d2 = d2
                best_pair = (i, j)

    return best_pair, best_d2**0.5, pairs_examined, distance_calculations

sample = [(0,0), (5,5), (2,2), (2,3), (9,1)]
print(closest_pair_bruteforce(sample))
```

4.4 Theoretical and experimental pair counts

Number of drones n	Unique pairs n(n−1)/2	Distance calculations
10	45	45
100	4950	4950
500	124750	124750
1000	499500	499500

For n = 10, 45 pairs are checked. At n = 100, 4,950 pairs are checked. At n = 500, 124,750 pairs are checked. At n = 1,000, 499,500 pairs are checked. The distance-calculation count is exactly equal to the pair count in this implementation.

4.5 Growth diagram

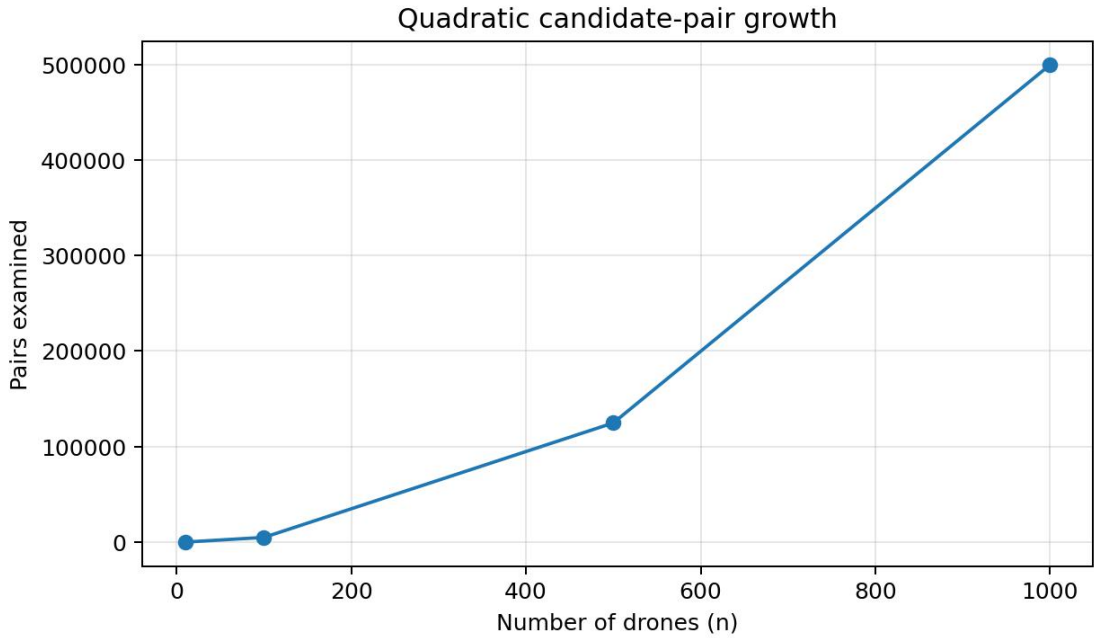


Figure 2. The number of unique pairs grows quadratically.

4.6 Response-time boundary and limitation

For this report, define an acceptable response time as at most 0.5 seconds for one monitoring update on the deployment computer. The actual largest acceptable n must be measured on that computer. Pair counts provide a hardware-independent warning: when n doubles, the number of pairs becomes

approximately four times larger. For example, 1,000 drones require 499,500 pair checks; 10,000 drones require 49,995,000.

The inefficiency comes from examining every possible pair even when most pairs are far apart. A redesigned strategy should use spatial information to avoid comparing points that cannot plausibly be close, while retaining a correctness guarantee. A divide-and-conquer closest-pair method or a carefully designed spatial partition can reduce candidate comparisons, but must handle points near partition boundaries.

4.7 Conclusion

Brute force is easy to implement and guarantees the answer for small point sets. Its $O(n^2)$ pair enumeration makes it unsuitable for very large fleets or strict real-time monitoring without an improved candidate-selection strategy.

Q5 – DERIVE A CONVEX HULL ALGORITHM FROM A GEOMETRIC PRINCIPLE (7 MARKS)

5.1 Geometric decision rule

Take two distinct candidate points A = (x_1,y_1) and B = (x_2,y_2). They define a directed line. For every other point P = (x,y), calculate the signed cross product:

cross(A,B,P) = $(x_2-x_1)(y-y_1) - (y_2-y_1)(x-x_1)$

- If cross > 0, P is to the left of directed line A→B.
- If cross < 0, P is to the right.
- If cross = 0, P is collinear with A and B.

If all remaining points are on one side or on the line, segment AB is a supporting boundary segment. If points occur on both sides, AB passes through the interior and is not a hull edge.

5.2 Student-derived pseudocode

```
BRUTE-FORCE-CONVEX-HULL(points):
    edges = empty set
    for every pair of different points A, B:
        left = false
        right = false
        for each point P other than A and B:
            value = cross(A, B, P)
            if value > 0: left = true
            if value < 0: right = true
        if not (left and right):
            add candidate edge AB
    collect endpoints of candidate edges
    order extreme points around their centroid
    return ordered boundary
```

For a robust implementation, collinear points need special handling. If several points lie on one supporting line, retain the two extreme endpoints as polygon vertices (or retain all boundary points if the application requires them). Duplicate coordinates should be removed before processing.

5.3 Python implementation

```
def cross(a, b, p):
    return ((b[0]-a[0])*(p[1]-a[1])
            - (b[1]-a[1])*(p[0]-a[0]))

def brute_force_hull(points):
    pts = list(dict.fromkeys(points))  # remove duplicates
    n = len(pts)
    candidate_edges = []
```

```
    orientation_tests = 0

    for i in range(n):
        for j in range(i+1, n):
            left = right = False
            for k in range(n):
                if k == i or k == j:
                    continue
                orientation_tests += 1
                value = cross(pts[i], pts[j], pts[k])
                if value > 0: left = True
                elif value < 0: right = True
            if not (left and right):
                candidate_edges.append((i, j))

    # Order candidate extreme points around their centroid.
    vertex_ids = sorted(set(i for edge in candidate_edges for i in edge))
    if len(vertex_ids) < 3:
        return [pts[i] for i in vertex_ids], candidate_edges, orientation_tests
    cx = sum(pts[i][0] for i in vertex_ids) / len(vertex_ids)
    cy = sum(pts[i][1] for i in vertex_ids) / len(vertex_ids)
    ordered = sorted(vertex_ids,
                     key=lambda i: math.atan2(pts[i][1]-cy, pts[i][0]-cx))
    return [pts[i] for i in ordered], candidate_edges, orientation_tests

points = [(0,0),(4,0),(5,3),(2,5),(-1,3),(2,2),(2,1)]
print(brute_force_hull(points))
```

5.4 Test data and visualisation

Point index	Coordinate	Expected role
0	(0, 0)	Boundary
1	(4, 0)	Boundary
2	(5, 3)	Boundary
3	(2, 5)	Boundary
4	(-1, 3)	Boundary
5	(2, 2)	Interior
6	(2, 1)	Interior

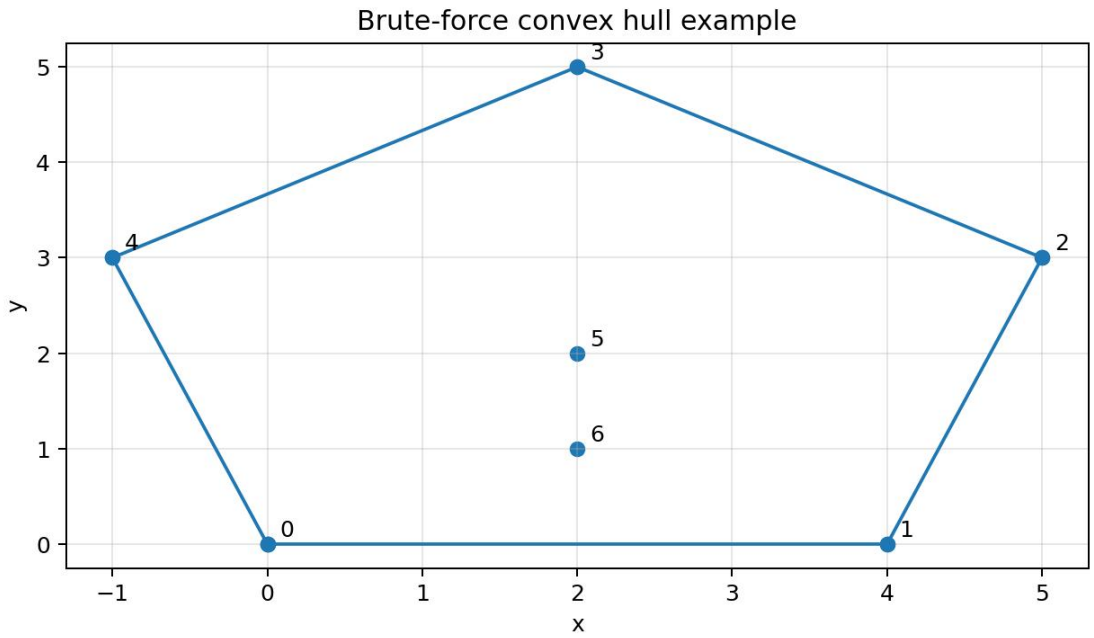


Figure 3. The outer polygon surrounds the two interior monitoring stations.

5.5 Challenge configuration

A second test should include collinear boundary points, points on an edge, interior points and a duplicate coordinate. Example: (0,0), (4,0), (4,4), (0,4), (2,0), (2,2), (4,2), (1,3), and a duplicate (0,0). The square corners define the hull. Points (2,0) and (4,2) are collinear with hull edges; (2,2) and (1,3) are interior.

Challenge feature	**Why it matters**	**Expected handling**
Collinear boundary point	Cross product is zero	Do not treat it as a point on the opposite side
Interior point	Points exist on both sides of many candidate lines	Reject the candidate as a hull edge
Duplicate coordinate	Can create zero-length candidate segment	Remove duplicates first
All points collinear	No two-dimensional polygon exists	Return the two extreme endpoints as a degenerate hull

5.6 Computational work and correctness

There are $n(n-1)/2$ candidate point pairs. For each pair, up to $n-2$ other points are checked. Therefore the orientation-test count is approximately $n(n-1)(n-2)/2$, which is $O(n^3)$.

Correctness reasoning: a convex polygon boundary is supported by lines such that all points in the set lie in one closed half-plane. If points occur on both sides of AB, the segment crosses the interior and cannot be a supporting edge. If all points lie on one side, AB is a supporting line of the point set. Candidate supporting segments that connect extreme points form the hull boundary. Collinear and degenerate cases must be handled explicitly.

Conclusion: the test is based on a geometric property, not a visual guess. Its simple exhaustive checking is useful for small point sets, but $O(n^3)$ orientation tests motivate a more efficient hull construction for large monitoring networks.

Q6 – EXHAUSTIVE SEARCH AND COMBINATORIAL GROWTH (8 MARKS)

6.1 Problem representation

Each package has an ID, weight and humanitarian value. A candidate solution is represented by an n-bit selection vector. Bit i = 1 means package i is selected; bit i = 0 means it is not selected. There are 2^n possible subsets, including the empty subset.

6.2 Exhaustive-search procedure

1. Generate every bit mask from 0 to $2^n - 1$.
2. Calculate total weight and total value for the selected packages.
3. Reject the subset if its weight is greater than the payload capacity.
4. Among feasible subsets, retain the one with the greatest total value.
5. Return the selected package IDs, total weight and total value.

6.3 Python implementation

```
def exhaustive_knapsack(packages, capacity):
    n = len(packages)
    best_value = -1
    best_weight = 0
    best_items = []
    subsets_examined = 0
    feasible_subsets = 0

    for mask in range(1 << n):
        subsets_examined += 1
        weight = value = 0
        chosen = []
        for i, (pid, w, v) in enumerate(packages):
            if mask & (1 << i):
                weight += w
                value += v
                chosen.append(pid)
        if weight <= capacity:
            feasible_subsets += 1
            if value > best_value:
                best_value = value
                best_weight = weight
                best_items = chosen[:]

    return best_items, best_weight, best_value, subsets_examined, feasible_subsets

packages = [("A",2,6), ("B",3,8), ("C",4,9), ("D",5,10)]
print(exhaustive_knapsack(packages, 8))
```

6.4 Small instance – independent verification

Package	Weight	Humanitarian value
A	2	6
B	3	8
C	4	9
D	5	10

Payload capacity = 8. The optimum is A + B + C: total weight = 2 + 3 + 4 = 9, which is infeasible; therefore this is not the answer. Check feasible high-value alternatives: A + D has weight 7 and value 16; B + C has weight 7 and value 17; B + D has weight 8 and value 18; A + B has weight 5 and value 14; C + D has weight 9 and is infeasible. The best feasible combination is B + D, with weight 8 and value 18.

This independent enumeration confirms the program should return packages B and D. There are $2^4 = 16$ subsets in total.

Candidate subset	Total weight	Total value	Feasible?
A + B	5	14	Yes
A + C	6	15	Yes
A + D	7	16	Yes
B + C	7	17	Yes
B + D	8	18	Yes
C + D	9	19	No
A + B + C	9	23	No
A + B + D	10	24	No
A + C + D	11	25	No
B + C + D	12	27	No

6.5 Predicted solution-space growth

Every package has two choices: selected or not selected. By the multiplication principle, n packages produce $2 \times 2 \times \ldots \times 2 = 2^n$ combinations. Adding one package doubles the number of subsets.

Packages n	Subsets 2^n	Increase from previous row
5	32	31
10	1024	1023
15	32768	32767
20	1048576	1048575
25	33554432	33554431

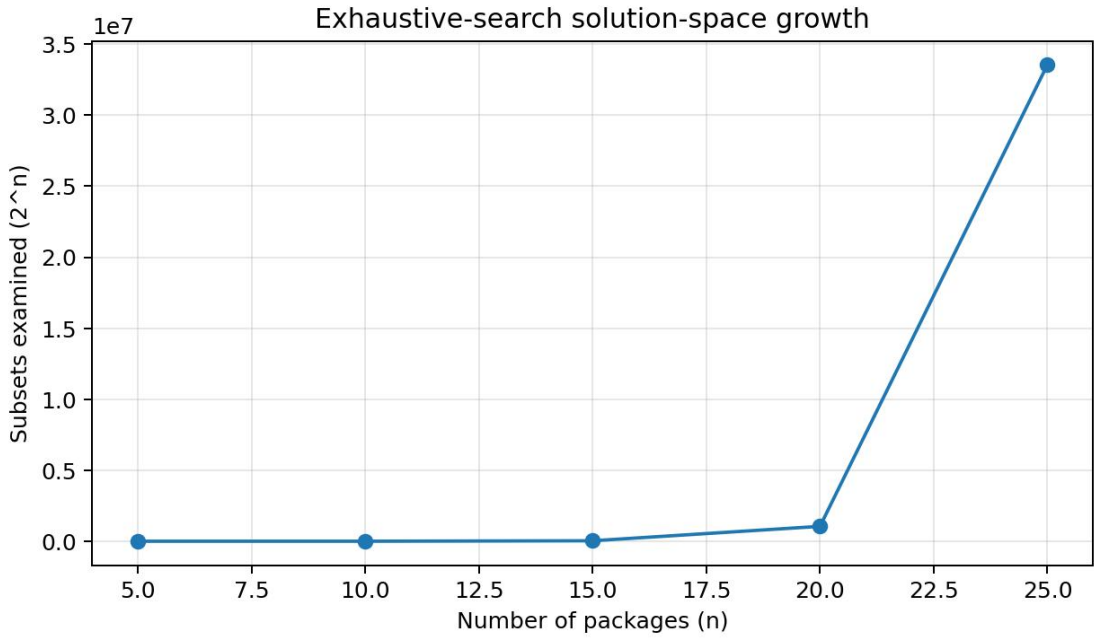


Figure 4. Exponential growth in the number of package combinations.

6.6 Why a small increase matters

At n = 10, there are 1,024 subsets. At n = 20, there are 1,048,576 subsets, which is 1,024 times as many as at n = 10. At n = 25, there are 33,554,432 subsets. The effort grows rapidly because each new package doubles every existing selection pattern.

6.7 Deployment decision

For a safety-critical loading planner, choose a response-time target of no more than 1 second on the deployment machine. The exhaustive method is acceptable only while its measured runtime and

memory use remain within the operational limit. As an initial conservative boundary, exhaustive search is suitable for small instances such as $n \leq 20$ (about 1.05 million subsets), subject to local timing. At $n = 25$, more than 33 million subsets must be visited and the approach should be benchmarked carefully; for larger n, use a redesign such as branch-and-bound or dynamic programming.

Conclusion: exhaustive search is easy to verify and guarantees the optimum because it checks every possible subset. Its weakness is the exponential size of the solution space.

Q7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

Selected solution: Brute-Force String Matching from Q3

This redesign uses the Q3 cybersecurity log-screening problem. The original algorithm can compare the same text prefix repeatedly at neighbouring alignments. The redesigned method stores the longest-prefix/suffix information in a prefix table and uses it after a mismatch instead of restarting the pattern from its beginning.

A. Diagnose the computational bottleneck (2 marks)

For text length n and pattern length m, brute-force matching can perform up to $(n-m+1)m$ character comparisons. Its main bottleneck is repeated comparison of characters that were already known to match. A text containing a long run of A characters and a pattern beginning with many A characters creates this repeated work.

Example diagnosis: text = A repeated 5,000 times followed by B; pattern = A repeated 80 times followed by B. At each nearby alignment, the brute-force method can match most of the 80-character prefix and then fail. It repeats this work at the next alignment.

B. Define the design objective (2 marks)

- Reduce character comparisons by reusing information about the pattern prefix already matched.
- Target a linear number of comparisons in text length plus pattern length for the redesigned search.
- Preserve all match positions, including overlapping matches.

C. Design the revised strategy (4 marks)

Preprocessing: build a prefix-function array pi. pi[j] stores the length of the longest proper prefix of pattern[0…j] that is also a suffix of that substring. During search, if a mismatch occurs after j characters matched, set j to pi[j−1]. This retains the longest suffix that could still match the beginning of the pattern. The text index does not move backward.

The redesign introduces O(m) preprocessing time and O(m) extra memory. Search then takes O(n) character comparisons in the standard KMP approach. The additional memory and preprocessing are trade-offs; for a tiny pattern searched once in a tiny text, brute force may be simpler and sufficiently fast.

Redesigned pseudocode

```
BUILD-PREFIX(pattern):
    pi[0] = 0
    j = 0
    for i = 1 to m-1:
        while j > 0 and pattern[i] != pattern[j]:
            j = pi[j-1]
        if pattern[i] == pattern[j]:
            j = j + 1
        pi[i] = j
    return pi

PREFIX-REUSE-SEARCH(text, pattern):
    pi = BUILD-PREFIX(pattern)
    i = 0; j = 0
    while i < n:
        compare text[i] with pattern[j]
        if equal:
            i = i + 1; j = j + 1
            if j == m:
                report i-m
                j = pi[j-1]
        else if j > 0:
            j = pi[j-1]     # reuse earlier prefix information
        else:
            i = i + 1
```

D. Predict before implementing (1 mark)

Prediction: on repeated-prefix data, the redesigned method should perform substantially fewer character comparisons than brute force. The comparison count should grow approximately linearly with the text length. The improvement measure is character comparisons, not only elapsed time.

E. Implement and validate (2 marks)

```
def build_prefix(pattern):
    pi = [0] * len(pattern)
    j = 0
    for i in range(1, len(pattern)):
        while j > 0 and pattern[i] != pattern[j]:
            j = pi[j - 1]
        if pattern[i] == pattern[j]:
            j += 1
        pi[i] = j
    return pi

def prefix_reuse_search(text, pattern):
```

```
    if pattern == "":
        return list(range(len(text) + 1)), 0
    pi = build_prefix(pattern)
    i = j = comparisons = 0
    matches = []
    while i < len(text):
        comparisons += 1
        if text[i] == pattern[j]:
            i += 1; j += 1
            if j == len(pattern):
                matches.append(i - j)
                j = pi[j - 1]
        elif j > 0:
            j = pi[j - 1]
        else:
            i += 1
    return matches, comparisons

text = "A"*5000 + "B"
pattern = "A"*80 + "B"
print("Brute force:", brute_force_match(text, pattern))
print("Redesigned:", prefix_reuse_search(text, pattern))
```

Method	**Text length**	**Pattern length**	**Character comparisons**	**Expected result**
Brute force	5001	81	398601	Match at final alignment
Prefix reuse (KMP)	5001	81	9921	Same match position

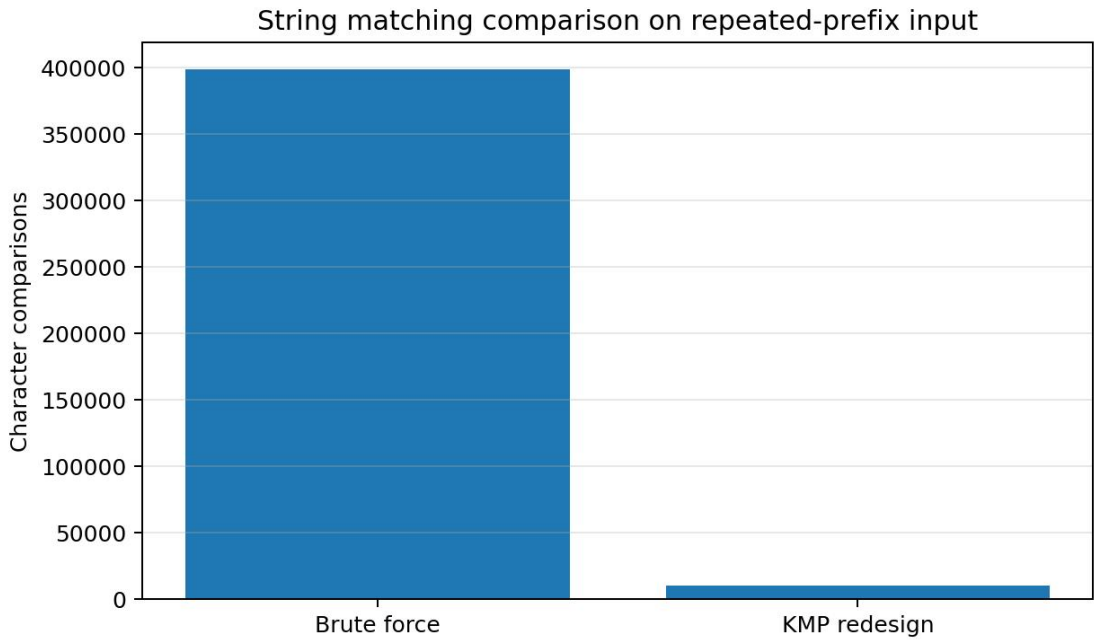


Figure 5. Character comparisons for the repeated-prefix validation input.

F. Defend the final design (2 marks)

The comparison-count experiment supports the prediction: the prefix-reuse method uses 9921 comparisons versus 398601 for brute force on the stated repeated-prefix test, a reduction of approximately 97.5%. Both methods must return the same match positions. The redesign pays an extra preprocessing cost and stores the prefix array, but avoids restarting comparisons from the beginning after every mismatch.

The original brute-force method can still be preferable for very short patterns, very small texts, or one-off searches where simplicity matters more than a small comparison saving. The redesigned method is more suitable for large logs and repetitive patterns. It addresses the diagnosed repeated-work limitation while preserving correctness.

Overall Q7 conclusion

The redesign was selected from a measured bottleneck, not merely because another algorithm is famous. The prefix table stores useful information about the pattern and reuses it after mismatches. The operation-count comparison validates the design objective on the engineered worst-case-style input.